

# quantfin-toolkit

Notes on the math and the code

Jun Shen

Jan 20, 2026

These are my own working notes on `quantfin-toolkit`, the small Python library I built out of my CQF and PhD coursework material. The point of writing this up properly, rather than just leaving comments scattered across the code, is that I wanted one place where the derivation, the design decision, and the actual implementation sit next to each other – partly so I don’t forget my own reasoning six months from now, and partly so I have something concrete to walk an interviewer through instead of just saying “I built a pricing library” and hoping they take my word for it.

Every section follows the same pattern: what the method is and where it comes from, the derivation (I’ve tried not to skip steps that actually matter), how I implemented it, and a couple of things that tripped me up or that I deliberately simplified. Where it’s useful I’ve also included the exact command to run the corresponding example script, so this document doubles as a runbook.

The full source, the test suite, and the example scripts are in the repository this document sits alongside. Everything below refers to real file paths and real function names, not pseudocode.

## Contents

|          |                                                                                      |           |
|----------|--------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Before you start</b>                                                              | <b>3</b>  |
| 1.1      | Getting it running . . . . .                                                         | 3         |
| 1.2      | How the package is laid out . . . . .                                                | 3         |
| <b>2</b> | <b>Numerical foundations — <code>quantfin/_numerics.py</code></b>                    | <b>4</b>  |
| 2.1      | Why this file exists at all . . . . .                                                | 4         |
| 2.2      | Standard normal CDF and PDF . . . . .                                                | 4         |
| 2.3      | Inverse normal CDF (the $z$ -score function) . . . . .                               | 4         |
| 2.4      | Chi-square CDFs at 1 and 2 degrees of freedom . . . . .                              | 5         |
| 2.5      | Bisection solver . . . . .                                                           | 5         |
| <b>3</b> | <b>Time value of money — <code>quantfin/time_value.py</code></b>                     | <b>7</b>  |
| <b>4</b> | <b>Stochastic process simulation — <code>quantfin/stochastic_processes.py</code></b> | <b>8</b>  |
| 4.1      | Wiener process . . . . .                                                             | 8         |
| 4.2      | Geometric Brownian motion . . . . .                                                  | 8         |
| 4.3      | Ornstein-Uhlenbeck / Vasicek . . . . .                                               | 8         |
| <b>5</b> | <b>Fixed income — <code>quantfin/fixed_income.py</code></b>                          | <b>10</b> |
| 5.1      | Day count conventions . . . . .                                                      | 10        |

|           |                                                                                           |           |
|-----------|-------------------------------------------------------------------------------------------|-----------|
| 5.2       | Yield curve . . . . .                                                                     | 10        |
| 5.3       | Clean vs. dirty price and accrued interest . . . . .                                      | 11        |
| 5.4       | Yield to maturity, duration, convexity . . . . .                                          | 11        |
| 5.5       | Binomial short-rate lattice and callable/putable bonds . . . . .                          | 12        |
| <b>6</b>  | <b>Options — quantfin/options/</b>                                                        | <b>14</b> |
| 6.1       | Black-Scholes-Merton — <code>black_scholes.py</code> . . . . .                            | 14        |
| 6.2       | Cox-Ross-Rubinstein binomial tree — <code>binomial_tree.py</code> . . . . .               | 14        |
| 6.3       | Finite difference PDE solvers — <code>finite_difference.py</code> . . . . .               | 15        |
| 6.4       | Monte Carlo — <code>monte_carlo.py</code> . . . . .                                       | 16        |
| <b>7</b>  | <b>Covariance estimation and normality diagnostics</b>                                    | <b>18</b> |
| 7.1       | Why the sample covariance matrix isn't good enough — <code>covariance.py</code> . . . . . | 18        |
| 7.2       | Ledoit-Wolf shrinkage . . . . .                                                           | 18        |
| 7.3       | PCA-based statistical factor model . . . . .                                              | 18        |
| 7.4       | Eigenvalue diagnostics . . . . .                                                          | 19        |
| 7.5       | ECDF, Q-Q plot, Jarque-Bera — <code>diagnostics.py</code> . . . . .                       | 19        |
| <b>8</b>  | <b>Portfolio construction — quantfin/portfolio.py</b>                                     | <b>21</b> |
| 8.1       | Why everything here is closed-form . . . . .                                              | 21        |
| 8.2       | The one function everything reduces to . . . . .                                          | 21        |
| 8.3       | Minimum variance and mean-variance (with transaction costs) . . . . .                     | 21        |
| 8.4       | Maximum Sharpe ratio (tangency portfolio) . . . . .                                       | 22        |
| 8.5       | Efficient frontier . . . . .                                                              | 22        |
| 8.6       | Risk decomposition: MCTR and CCTR . . . . .                                               | 22        |
| <b>9</b>  | <b>Risk and model validation — quantfin/risk/</b>                                         | <b>24</b> |
| 9.1       | Value at Risk and Expected Shortfall — <code>value_at_risk.py</code> . . . . .            | 24        |
| 9.2       | VaR backtesting — <code>backtesting.py</code> . . . . .                                   | 24        |
| 9.3       | CAPM — <code>capm.py</code> . . . . .                                                     | 26        |
| <b>10</b> | <b>Running everything and the test suite</b>                                              | <b>27</b> |
| 10.1      | Example scripts, all in one place . . . . .                                               | 27        |
| 10.2      | Testing methodology . . . . .                                                             | 27        |

# 1 Before you start

## 1.1 Getting it running

```
git clone https://github.com/<your-username>/quantfin-toolkit.git
cd quantfin-toolkit
pip install -r requirements.txt      # numpy, pandas, matplotlib, python-dateutil
python -m unittest discover -s tests # 142 tests, should all pass
PYTHONPATH=. python examples/run_option_pricing.py # e.g.
```

No SciPy, no pytest. That wasn't an accident I'm apologizing for – Section 2 explains why I kept it that way even once SciPy was available to me again.

## 1.2 How the package is laid out

```
quantfin/
  _numerics.py          normal CDF/PDF/inverse, chi-square CDFs, bisection solver
  time_value.py         discrete/continuous compounding
  stochastic_processes.py Wiener, GBM, OU, Vasicek
  fixed_income.py       day count, yield curve, bonds, callable/puttable lattice
  covariance.py         sample cov, Ledoit-Wolf shrinkage, PCA factor model
  diagnostics.py        ECDF, QQ-plot, Jarque-Bera
  portfolio.py          mean-variance optimization, frontier, MCTR/CCTR
  options/
    black_scholes.py, binomial_tree.py, finite_difference.py, monte_carlo.py
  risk/
    value_at_risk.py, backtesting.py, capm.py
  tests/               one file per module, unittest
  examples/            one runnable script per topic
```

I'll go through it roughly in dependency order: the numerical primitives everything else sits on, then time value of money, stochastic processes, fixed income, options, covariance estimation, portfolio construction, and finally the risk / model validation layer, which is really the part I care most about and the part that ties everything else together.

## 2 Numerical foundations — quantfin/\_numerics.py

### 2.1 Why this file exists at all

Almost every quant library leans on SciPy for the normal distribution, the chi-square distribution, and a root finder. I didn't want that dependency here, for a reason that's more about the exercise than the constraint: if I'm going to put "model validation" on my CV, I want to be able to say I know what `scipy.stats.norm.cdf` is actually doing under the hood, not just that I know how to call it. So this file reimplements the handful of things the rest of the package needs, each with a closed form or a well-documented numerical approximation.

### 2.2 Standard normal CDF and PDF

The PDF has a closed form:

$$\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

and the CDF is its integral, which doesn't have an elementary closed form on its own but can be written exactly in terms of the error function:

$$\Phi(x) = \int_{-\infty}^x \phi(u) du = \frac{1}{2} \left[ 1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right) \right]$$

Python's standard library ships `math.erf`, which is itself a high-accuracy numerical implementation, so this isn't an approximation on my end – it's exact up to floating point precision.

```

1 def normal_cdf(x: float) -> float:
2     """Standard normal CDF, computed exactly via the error function."""
3     return 0.5 * (1.0 + math.erf(x / math.sqrt(2.0)))
4
5 def normal_pdf(x: float) -> float:
6     """Standard normal PDF."""
7     return math.exp(-0.5 * x * x) / math.sqrt(2.0 * math.pi)

```

### 2.3 Inverse normal CDF (the $z$ -score function)

This one doesn't have a closed form at all –  $\Phi^{-1}(p)$  has to be solved for numerically. I used Peter Acklam's rational approximation to get a first guess accurate to roughly  $1.15 \times 10^{-9}$ , then polished it with Newton's method against the exact CDF/PDF above:

$$x_{k+1} = x_k - \frac{\Phi(x_k) - p}{\phi(x_k)}$$

which is just Newton-Raphson applied to solving  $\Phi(x) - p = 0$ . Two or three iterations from Acklam's starting point are enough to hit machine precision, because Newton's method converges quadratically once you're close.

```

1 def normal_ppf(p: float, tol: float = 1e-12, max_iter: int = 100) -> float:
2     if not 0.0 < p < 1.0:
3         raise ValueError(f"p must be in the open interval (0, 1), got {p}")
4     # ... Acklam's rational approximation gives the starting guess x ...
5     for _ in range(max_iter):
6         error = normal_cdf(x) - p
7         if abs(error) < tol:
8             break

```

```

9         x -= error / normal_pdf(x)
10    return x

```

I checked this against known reference values before trusting it anywhere else in the package:  $z_{0.95} = 1.6449$ ,  $z_{0.99} = 2.3263$ , both match the textbook numbers to four decimal places, which is what `tests/test_numerics.py` actually asserts.

## 2.4 Chi-square CDFs at 1 and 2 degrees of freedom

I only need the chi-square CDF for the VaR backtests in Section 9.2, and only at  $df = 1$  (Kupiec, Christoffersen independence) and  $df = 2$  (Christoffersen conditional coverage, Jarque-Bera). Both happen to have nice closed forms in terms of things I already have.

For  $df = 1$ : if  $Z \sim N(0, 1)$  then  $Z^2 \sim \chi_1^2$ , and

$$P(\chi_1^2 \leq x) = P(-\sqrt{x} \leq Z \leq \sqrt{x}) = 2\Phi(\sqrt{x}) - 1$$

For  $df = 2$ , the chi-square distribution is exactly an exponential distribution with rate  $1/2$ :

$$P(\chi_2^2 \leq x) = 1 - e^{-x/2}$$

```

1 def chi2_cdf_df1(x: float) -> float:
2     if x < 0:
3         return 0.0
4     return 2.0 * normal_cdf(math.sqrt(x)) - 1.0
5
6 def chi2_cdf_df2(x: float) -> float:
7     if x < 0:
8         return 0.0
9     return 1.0 - math.exp(-x / 2.0)

```

I checked these against the standard critical values too:  $\chi_{1,0.95}^2 = 3.841$  and  $\chi_{2,0.95}^2 = 5.991$  both come out right, which is really just a sanity check that I derived the closed forms correctly.

## 2.5 Bisection solver

Used in exactly one place: solving for a bond's yield to maturity given its price (Section 5). The price-as-a-function-of-yield relationship is smooth and monotonically decreasing, so a plain bisection search is the boring, reliable choice – I didn't want a fancier solver (secant, Newton) that could misbehave if it's ever handed a bad starting bracket.

```

1 def bisection_solve(func, lower, upper, tol=1e-8, max_iter=200) -> float:
2     f_lower, f_upper = func(lower), func(upper)
3     if f_lower * f_upper > 0.0:
4         raise ValueError("bisection_solve requires opposite signs at the endpoints")
5     for _ in range(max_iter):
6         midpoint = 0.5 * (lower + upper)
7         f_mid = func(midpoint)
8         if abs(f_mid) < tol or (upper - lower) / 2.0 < tol:
9             return midpoint
10        if f_lower * f_mid < 0.0:
11            upper, f_upper = midpoint, f_mid
12        else:
13            lower, f_lower = midpoint, f_mid
14    return 0.5 * (lower + upper)

```

If I ever wanted to swap these out for SciPy, the call signatures were kept intentionally close to `scipy.stats.norm`, `scipy.stats.chi2`, and `scipy.optimize.brentq`, so it would be a near drop-in replacement. I just didn't see a reason to.

### 3 Time value of money — `quantfin/time_value.py`

This is the one module that's genuinely just four formulas, but everything downstream (discounting cash flows, pricing bonds, pricing options) is built on the distinction between discrete and continuous compounding, so it's worth being precise about it once here rather than re-deriving it in every later section.

Discrete compounding at rate  $r$  over  $n$  periods:

$$FV = PV (1 + r)^n \qquad PV = FV (1 + r)^{-n}$$

Continuous compounding, which is the limit as the compounding frequency  $m \rightarrow \infty$  of  $(1 + r/m)^{mt}$ :

$$\lim_{m \rightarrow \infty} \left(1 + \frac{r}{m}\right)^{mt} = e^{rt} \quad \Rightarrow \quad FV = PV e^{rt} \qquad PV = FV e^{-rt}$$

```

1 def future_value_discrete(present_value, rate, periods):
2     return present_value * (1.0 + rate) ** periods
3
4 def future_value_continuous(present_value, rate, time):
5     return present_value * math.exp(rate * time)
```

Everywhere else in the library that discounts a cash flow – the yield curve in `fixed_income.py`, the risk-neutral drift in the option pricers – uses continuous compounding, because that's the convention that makes forward rates additive across periods (Section 5) and matches the way the Black-Scholes PDE is usually written.

**Run it:** there's no standalone example script for this module since it's used internally everywhere else; `tests/test_time_value.py` exercises all four functions against hand-calculated values.

## 4 Stochastic process simulation — quantfin/stochastic\_processes.py

### 4.1 Wiener process

A standard Wiener process  $W(t)$  is defined by  $W(0) = 0$ , independent increments, and  $W(t) - W(s) \sim N(0, t - s)$  for  $t > s$ . Discretizing on a grid with step  $\Delta t$ , each increment is

$$\Delta W_i = W(t_{i+1}) - W(t_i) \sim N(0, \Delta t) \implies \Delta W_i = \sqrt{\Delta t} \cdot Z_i, \quad Z_i \sim N(0, 1)$$

and the path is just the cumulative sum of the increments. This is exact – there’s no discretization error in simulating a Wiener process this way, since Brownian motion’s increments really are independent and normally distributed at any resolution.

```
1 rng = np.random.default_rng(seed)
2 increments = rng.normal(loc=0.0, scale=np.sqrt(dt), size=n_steps)
3 w = np.zeros(n_steps + 1)
4 w[1:] = np.cumsum(increments)
```

### 4.2 Geometric Brownian motion

GBM is the SDE behind Black-Scholes:

$$dS = \mu S dt + \sigma S dW$$

Applying Itô’s lemma to  $\ln S$  (this is the derivation I always want to have ready in an interview, since it’s the one step people actually probe on): let  $f(S) = \ln S$ , so  $f'(S) = 1/S$  and  $f''(S) = -1/S^2$ . Itô’s lemma gives

$$d(\ln S) = (f'(S)\mu S + \tfrac{1}{2}f''(S)\sigma^2 S^2) dt + f'(S)\sigma S dW = (\mu - \tfrac{1}{2}\sigma^2) dt + \sigma dW$$

which has constant coefficients, so it integrates directly:

$$\ln S(t) - \ln S(0) = (\mu - \tfrac{1}{2}\sigma^2) t + \sigma W(t) \implies S(t) = S(0) \exp[(\mu - \tfrac{1}{2}\sigma^2) t + \sigma W(t)]$$

This is the closed-form solution, so I can simulate GBM exactly (no Euler discretization error) just by simulating  $W(t)$  and plugging it in:

```
1 path = s0 * np.exp((mu - 0.5 * sigma ** 2) * time_grid + sigma * w)
```

The  $-\frac{1}{2}\sigma^2$  term is the one people forget when they eyeball this formula – it’s not a typo, it’s exactly what makes  $E[S(t)] = S(0)e^{\mu t}$  come out right (Jensen’s inequality:  $E[e^X] > e^{E[X]}$  for a random  $X$ , and the  $-\frac{1}{2}\sigma^2 t$  correction exactly cancels that gap).

### 4.3 Ornstein-Uhlenbeck / Vasicek

The OU process is the standard mean-reverting SDE:

$$dx = \theta(\mu - x) dt + \sigma dW$$

Unlike GBM this doesn’t have as convenient a form to simulate exactly along a grid without extra work, so I used the standard Euler-Maruyama discretization:

$$x_{i+1} = x_i + \theta(\mu - x_i)\Delta t + \sigma\sqrt{\Delta t} Z_i$$



```
1 for i in range(1, n_steps + 1):  
2     x[i] = x[i - 1] + theta * (mu - x[i - 1]) * dt + sigma * shocks[i - 1]
```

The Vasicek short-rate model,  $dr = \kappa(\theta - r) dt + \sigma dW$ , is literally the same equation with the variables renamed ( $\kappa \leftrightarrow \theta$ ,  $\theta \leftrightarrow \mu$ ,  $r \leftrightarrow x$ ). The tutorial material I originally worked from had two separate, nearly-identical implementations of this; `simulate_vasicek` here is just a thin renaming wrapper that calls `simulate_ornstein_uhlenbeck`, which is the kind of small cleanup that's easy to miss if you're translating formulas one-for-one instead of noticing they're the same process:

```
1 def simulate_vasicek(r0, kappa, theta, sigma, t, n_steps, seed=None):  
2     dt = t / n_steps  
3     rate_path = simulate_ornstein_uhlenbeck(  
4         x0=r0, theta=kappa, mu=theta, sigma=sigma, n_steps=n_steps, dt=dt, seed=seed  
5     )  
6     return time_grid, rate_path
```

**Run it:** no dedicated example script – these functions feed into the binomial short-rate lattice in the next section and are exercised directly in `tests/test_stochastic_processes.py` (checking, among other things, that GBM paths stay strictly positive and that a Wiener process's variance grows like  $t$ ).

## 5 Fixed income — quantfin/fixed\_income.py

This is the largest module in the package, and the one I spent the most time getting the small details right on, mostly because bond pricing is full of conventions (day counts, clean vs. dirty price) that are easy to get approximately right and subtly wrong.

### 5.1 Day count conventions

The three I implemented:

- **ACT/360**: actual calendar days between two dates, divided by 360. Common in money markets.
- **ACT/365**: actual days divided by 365. Common for government bonds in some markets.
- **30/360 (Bond Basis)**: pretends every month has 30 days. The rule I implemented (the standard “Bond Basis” variant) is: if the start day is 30 or 31, treat it as 30; if the start day is (now) 30 and the end day is 31, treat the end day as 30 too.

```

1 if convention == DayCountConvention.THIRTY_360:
2     d1, m1, y1 = start.day, start.month, start.year
3     d2, m2, y2 = end.day, end.month, end.year
4     d1 = min(d1, 30)
5     if d1 == 30:
6         d2 = min(d2, 30)
7     days = (y2 - y1) * 360 + (m2 - m1) * 30 + (d2 - d1)
8     return days / 360.0

```

I default coupon accrual to 30/360 (the market convention for most fixed-rate corporate and government bonds) but discount cash flows using ACT/365 against the yield curve – these are genuinely different conventions used for different purposes in practice, not an inconsistency in my code.

### 5.2 Yield curve

I model the curve as a set of continuously-compounded zero rates at given tenors, linearly interpolated in between and held flat outside the given range:

```

1 def spot_rate(self, t: float) -> float:
2     return float(np.interp(t, self.tenors, self.rates))
3
4 def discount_factor(self, t: float) -> float:
5     if t == 0:
6         return 1.0
7     return math.exp(-self.spot_rate(t) * t)

```

The forward rate between  $t_1$  and  $t_2$  comes from the no-arbitrage condition that compounding at the spot rate to  $t_1$  and then at the forward rate to  $t_2$  has to match compounding at the spot rate straight to  $t_2$ :

$$e^{-y(t_1)t_1} \cdot e^{-f(t_1,t_2)(t_2-t_1)} = e^{-y(t_2)t_2} \implies f(t_1,t_2) = \frac{y(t_2)t_2 - y(t_1)t_1}{t_2 - t_1}$$

```

1 def forward_rate(self, t1: float, t2: float) -> float:
2     return (self.spot_rate(t2) * t2 - self.spot_rate(t1) * t1) / (t2 - t1)

```

This forward-rate identity is also what makes the short-rate lattice in Section 5.5 reprice a plain bond back to (essentially) the curve-discounted price even without full Ho-Lee style calibration – more on that below.

### 5.3 Clean vs. dirty price and accrued interest

The *dirty price* is the actual present value of all remaining cash flows – what you’d pay to buy the bond today:

$$P_{\text{dirty}} = \sum_i CF_i \cdot e^{-y(t_i)t_i}$$

The *clean price* is the dirty price minus accrued interest – the price that’s actually quoted in the market, because otherwise the quoted price would jump downward every time a coupon is paid, which is confusing to look at on a screen:

$$P_{\text{clean}} = P_{\text{dirty}} - AI$$

Accrued interest is the fraction of the current coupon period that has already elapsed, times the coupon:

$$AI = C \cdot \frac{\tau(\text{last coupon, settlement})}{\tau(\text{last coupon, next coupon})}$$

```

1 def accrued_interest(self, settlement_date: date) -> float:
2     if settlement_date >= self.maturity_date:
3         return 0.0
4     # strict ">": settlement landing exactly on a coupon date means that
5     # coupon has just been paid, so the next accrual period hasn't started
6     prev_date = self.issue_date
7     next_date = None
8     for coupon_date in self.coupon_dates():
9         if coupon_date > settlement_date:
10             next_date = coupon_date
11             break
12     prev_date = coupon_date
13     if next_date is None:
14         return 0.0
15     period_elapsed = year_fraction(prev_date, settlement_date, self.day_count)
16     full_period = year_fraction(prev_date, next_date, self.day_count)
17     return self._coupon_amount() * (period_elapsed / full_period)

```

The strict > on line 7 above is not cosmetic. My first version used >=, and it failed a test I’d written (`test_accrued_interest_is_zero_right_at_a_coupon_date`) that checks accrued interest is exactly zero when settlement lands precisely on a coupon date. With >=, a coupon date equal to the settlement date gets treated as the still-pending “next” coupon, which computes a full period’s worth of accrual instead of zero. It’s a one-character bug, and it’s exactly the kind of off-by-one that a real model validation review would (and should) catch.

### 5.4 Yield to maturity, duration, convexity

Yield to maturity is the flat rate  $y$  (compounded `frequency` times per year) that makes the discounted cash flows equal the observed price:

$$P = \sum_i \frac{CF_i}{\left(1 + \frac{y}{f}\right)^{f \cdot t_i}}$$

There's no closed form for  $y$  given  $P$ , so I solve it with the bisection solver from Section 2 on the error function  $P(y) - P_{\text{observed}}$ , which is smooth and strictly decreasing in  $y$  for a bond with positive cash flows.

Macaulay duration is the cash-flow-weighted average time to receipt, discounting each flow at the bond's own YTM:

$$D_{\text{Mac}} = \frac{\sum_i t_i \cdot PV_i}{\sum_i PV_i}, \quad PV_i = \frac{CF_i}{(1 + y/f)^{ft_i}}$$

Modified duration rescales this into a price sensitivity (the standard first-order approximation to  $-\frac{1}{P} \frac{dP}{dy}$ ):

$$D_{\text{mod}} = \frac{D_{\text{Mac}}}{1 + y/f}$$

Convexity is the second-order term, which comes from differentiating the price-yield relationship twice:

$$C = \frac{1}{P(1 + y/f)^2} \sum_i PV_i \cdot t_i \left( t_i + \frac{1}{f} \right)$$

```

1 def convexity(self, price, settlement_date):
2     ytm = self.yield_to_maturity(price, settlement_date)
3     flows = self._remaining_cash_flows(settlement_date)
4     weighted_convexity = total_pv = 0.0
5     for t, cf in flows:
6         pv = cf / (1.0 + ytm / self.frequency) ** (t * self.frequency)
7         weighted_convexity += pv * t * (t + 1.0 / self.frequency)
8         total_pv += pv
9     return weighted_convexity / (total_pv * (1.0 + ytm / self.frequency) ** 2)

```

Together,  $\Delta P \approx -P \cdot D_{\text{mod}} \cdot \Delta y + \frac{1}{2} P \cdot C \cdot (\Delta y)^2$  is the standard second-order Taylor approximation to how a bond's price moves with a small parallel shift in yield – duration gets the linear part right, convexity corrects for the fact that the price-yield curve is, well, convex.

## 5.5 Binomial short-rate lattice and callable/puttable bonds

To price a bond with embedded optionality (issuer can call it back, or holder can put it back, at a fixed price after some date), a single flat discount rate isn't enough – you need a model of how the short rate can move, because the decision to call or put depends on where rates are at that future date.

I built a simple recombining binomial tree of short rates. At step  $i$  (covering  $[i\Delta t, (i+1)\Delta t]$ ), node  $j$  has rate

$$r_{i,j} = \underbrace{\text{drift}_i}_{\text{forward rate off the curve}} + (2j - i) \cdot \sigma \sqrt{\Delta t}$$

```

1 def build(self) -> List[List[float]]:
2     rates = []
3     for i in range(self.n_steps):
4         t1, t2 = i * self.dt, (i + 1) * self.dt
5         drift = self.curve.spot_rate(t2) if t1 == 0 else self.curve.forward_rate(t1, t2)
6         spread = self.volatility * math.sqrt(self.dt)
7         level = [drift + (2 * j - i) * spread for j in range(i + 1)]
8         rates.append(level)
9     return rates

```

**This is a deliberate simplification, and I say so directly in the module docstring rather than hoping nobody asks.** A production short-rate tree (Ho-Lee, Black-Derman-Toy) calibrates the drift at every single step so that discounting through the tree reprices every zero-coupon bond on the input curve *exactly*. What I built reads the drift straight off the curve's forward rates and doesn't iterate to correct for the small convexity effect that symmetric up/down noise introduces. In practice this tracks the curve closely – with zero volatility the two are exactly equal by the forward-rate telescoping identity – but it will drift slightly for high volatility or long horizons. Getting full calibration right is a project on its own, and it wasn't necessary to demonstrate that I understand backward induction and early-exercise mechanics, which is the actual point of this section.

Pricing is standard backward induction: at each node, the continuation value is the risk-neutral expectation of next period's value (equal up/down probabilities, since the up/down spread already encodes the risk premium through the volatility term), discounted one step, plus any cash flow paid at that node:

$$V_{i,j} = e^{-r_{i,j}\Delta t} \cdot \frac{1}{2} (V_{i+1,j} + V_{i+1,j+1}) + CF_{i,j}$$

Early exercise just clamps this value at each eligible node – a call caps the holder's value at the call price (the issuer won't let it be worth more than that to the holder), a put floors it (the holder won't accept less):

```
1 continuation = 0.5 * (next_values[j] + next_values[j + 1]) * math.exp(-r * self.dt)
2 node_value = continuation + cash_flows.get(i, 0.0)
3 if exercise == "call" and i >= exercise_start_step:
4     node_value = min(node_value, exercise_price)
5 elif exercise == "put" and i >= exercise_start_step:
6     node_value = max(node_value, exercise_price)
```

### Run it:

```
PYTHONPATH=. python examples/run_bond_pricing.py
PYTHONPATH=. python examples/run_callable_bond_pricing.py
```

The callable example prices a plain bond two ways – once via curve discounting, once via the same lattice with no early exercise – so the callable/putable prices are compared against a fair, apples-to-apples baseline rather than the curve-discounted number directly (a small but real distinction: discounting through a tree with symmetric noise is not bit-for-bit identical to deterministic curve discounting once volatility is nonzero).

## 6 Options — quantfin/options/

I implemented four ways to price the same vanilla European (or American, where it applies) option, deliberately, so I could cross-check them against each other: closed-form Black-Scholes, a binomial tree, a finite-difference PDE solver (three variants), and Monte Carlo. If you only ever implement one method, you never actually know if it's right; if you implement four independent ones and they agree to three decimal places, that's real evidence.

### 6.1 Black-Scholes-Merton — black\_scholes.py

The Black-Scholes PDE, for a non-dividend-paying underlying under the risk-neutral measure:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0$$

with terminal condition  $V(S, T) = \max(S - K, 0)$  for a call. The closed-form solution (with a continuous dividend yield  $q$ ) is

$$C = Se^{-qT}\Phi(d_1) - Ke^{-rT}\Phi(d_2), \quad P = Ke^{-rT}\Phi(-d_2) - Se^{-qT}\Phi(-d_1)$$

$$d_1 = \frac{\ln(S/K) + (r - q + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}$$

```
1 def call_price(spot, strike, time_to_maturity, risk_free_rate, volatility, dividend_yield=0.0):
2     d1, d2 = _d1_d2(spot, strike, time_to_maturity, risk_free_rate, volatility, dividend_yield)
3     return (spot * math.exp(-dividend_yield * time_to_maturity) * normal_cdf(d1)
4             - strike * math.exp(-risk_free_rate * time_to_maturity) * normal_cdf(d2))
```

The Greeks all fall out of differentiating this w.r.t. the relevant parameter. Delta and gamma are the ones worth having the derivation for on hand:

$$\Delta_{\text{call}} = e^{-qT}\Phi(d_1), \quad \Gamma = \frac{e^{-qT}\phi(d_1)}{S\sigma\sqrt{T}}$$

Gamma comes from differentiating delta w.r.t.  $S$  again; the reason it's the same for calls and puts (unlike delta and theta) is that put-call parity,  $C - P = Se^{-qT} - Ke^{-rT}$ , is linear in  $S$ , so its second derivative is zero – calls and puts must have identical gamma.

```
1 def gamma(spot, strike, time_to_maturity, risk_free_rate, volatility, dividend_yield=0.0):
2     d1, _ = _d1_d2(spot, strike, time_to_maturity, risk_free_rate, volatility, dividend_yield)
3     discount = math.exp(-dividend_yield * time_to_maturity)
4     return discount * normal_pdf(d1) / (spot * volatility * math.sqrt(time_to_maturity))
```

### 6.2 Cox-Ross-Rubinstein binomial tree — binomial\_tree.py

The tree approximates GBM with a discrete up/down process over  $n$  steps of size  $\Delta t = T/n$ :

$$u = e^{\sigma\sqrt{\Delta t}}, \quad d = 1/u$$

chosen so that the tree's variance matches the continuous process's variance in the limit  $n \rightarrow \infty$ . The risk-neutral probability of an up move is picked so the discounted expected stock price equals today's forward price:

$$p = \frac{e^{r\Delta t} - d}{u - d}$$

```

1 up = math.exp(volatility * math.sqrt(dt))
2 down = 1.0 / up
3 growth = math.exp(risk_free_rate * dt)
4 risk_neutral_prob = (growth - down) / (up - down)

```

I build all terminal payoffs at once with vectorized NumPy ( $\text{spot} \times u^j d^{n-j}$  for  $j = 0, \dots, n$ ), then walk backward, discounting the risk-neutral expectation one step at a time. For American exercise I take the elementwise max of the continuation value and the intrinsic value at every node – the holder exercises early whenever immediate exercise beats holding on:

```

1 for step in range(n_steps - 1, -1, -1):
2     values = discount * (risk_neutral_prob * values[1:] + (1 - risk_neutral_prob) * values
3         [-1])
4     if exercise == "american":
5         stock_prices_at_step = spot * (up ** j) * (down ** (step - j))
6         intrinsic = np.maximum(stock_prices_at_step - strike, 0.0) # call case
7         values = np.maximum(values, intrinsic)

```

With enough steps this converges to the Black-Scholes price for a European option – in `examples/run_option_pricing.py`, 500 steps gets within about half a cent of the closed form.

### 6.3 Finite difference PDE solvers — `finite_difference.py`

Instead of a tree or the closed form, this solves the Black-Scholes PDE directly on a  $(S, t)$  grid using finite differences for the derivatives. Let  $j$  index price levels and superscript  $n$  index the time level, with  $V_j^n \approx V(j\Delta S, n\Delta t)$ .

**Explicit scheme.** Approximate the time derivative forward and the space derivatives with central differences, all evaluated at time level  $n + 1$  (i.e. using values you already know, solving for the unknown level  $n$  by walking backward from maturity):

$$V_j^n = a_j V_{j-1}^{n+1} + b_j V_j^{n+1} + c_j V_{j+1}^{n+1}$$

$$a_j = \frac{1}{2} \Delta t (\sigma^2 j^2 - rj), \quad b_j = 1 - \Delta t (\sigma^2 j^2 + r), \quad c_j = \frac{1}{2} \Delta t (\sigma^2 j^2 + rj)$$

```

1 j = np.arange(1, self.n_price_steps)
2 a = 0.5 * self.dt * (self.volatility ** 2 * j ** 2 - self.risk_free_rate * j)
3 b = 1.0 - self.dt * (self.volatility ** 2 * j ** 2 + self.risk_free_rate)
4 c = 0.5 * self.dt * (self.volatility ** 2 * j ** 2 + self.risk_free_rate * j)
5 new_values[1:-1] = a * values[:-2] + b * values[1:-1] + c * values[2:]

```

This is cheap (no linear system to solve) but conditionally stable: it only produces a sane answer if  $\Delta t$  is small relative to  $(\Delta S)^2$ . The standard stability guideline (Wilmott, Howison & Dewynne) is

$$\Delta t \lesssim \frac{1}{\sigma^2 M^2}$$

where  $M$  is the number of price steps. I check this explicitly and warn rather than silently returning garbage:

```

1 max_stable_dt = 1.0 / (self.volatility ** 2 * self.n_price_steps ** 2)
2 if self.dt > max_stable_dt:
3     warnings.warn("explicit finite difference scheme is likely unstable for this grid ...")

```

**Implicit scheme.** Evaluate the space derivatives at the *unknown* time level  $n$  instead:

$$V_j^{n+1} = -\alpha_j V_{j-1}^n + \beta_j V_j^n - \gamma_j V_{j+1}^n$$

which means every interior point at level  $n$  depends on its unknown neighbors too – there’s no explicit formula, you have to solve a tridiagonal linear system for the whole vector of interior values at once, one system per time step:

```
1 interior_values = np.linalg.solve(matrix, rhs)
```

This costs more per step (a linear solve instead of a vector multiply) but is unconditionally stable – there’s no  $\Delta t$  vs.  $\Delta S$  tradeoff to worry about, which is exactly why it’s preferred in practice despite the extra cost.

**Crank-Nicolson.** The average of the explicit and implicit schemes – it’s second-order accurate in time (rather than first-order for the other two), and unconditionally stable like the implicit scheme, at the same per-step cost (still one tridiagonal solve). This is why it’s usually the default choice in practice, and why it’s the default `method` argument in `price()`.

The example script `run_fdm_pricing.py` deliberately runs the explicit scheme on a grid that violates the stability condition, to show what actually happens rather than just describing it: the price diverges to something on the order of  $10^{57}$ , nowhere near the Black-Scholes value. That’s not a bug – it’s the textbook failure mode explicit schemes have, and seeing the number blow up is a more convincing demonstration than a paragraph explaining why implicit methods exist.

**Run it:**

```
PYTHONPATH=. python examples/run_option_pricing.py # BS vs tree vs Monte Carlo, Greeks
PYTHONPATH=. python examples/run_fdm_pricing.py    # explicit/implicit/CN, incl. instability
demo
```

## 6.4 Monte Carlo — monte\_carlo.py

For a European option there’s no need to simulate the whole path – only the terminal price matters, and under risk-neutral GBM I already have the closed-form solution from Section 2:

$$S(T) = S(0) \exp \left[ \left( r - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} Z \right], \quad Z \sim N(0, 1)$$

The price is the discounted average payoff over many simulated terminal prices:

$$\hat{V} = e^{-rT} \cdot \frac{1}{N} \sum_{i=1}^N \max(S_i(T) - K, 0)$$

```
1 drift = (risk_free_rate - 0.5 * volatility ** 2) * time_to_maturity
2 diffusion = volatility * math.sqrt(time_to_maturity) * z
3 terminal_prices = spot * np.exp(drift + diffusion)
4 payoffs = np.maximum(terminal_prices - strike, 0.0)
5 price = float(np.mean(discount * payoffs))
```

I used antithetic variates as the variance reduction technique: pair every draw  $Z_i$  with its mirror  $-Z_i$ . Since both draws come from the same underlying normal distribution, this doesn’t bias the estimate, but because the payoff function isn’t symmetric around zero, the pair’s average has lower variance than two independent draws would – basically free variance reduction for the cost of one extra array concatenation:

```
1 z = rng.standard_normal(n_simulations)
2 if antithetic:
3     z = np.concatenate([z, -z])
```



The function also returns a standard error,  $SE = s/\sqrt{N}$  where  $s$  is the sample standard deviation of the discounted payoffs, so I always know how much simulation noise is left rather than just trusting a single point estimate.

**Run it:** same as above, `run_option_pricing.py` compares all three methods on one option and reports the Monte Carlo standard error alongside the price.

## 7 Covariance estimation and normality diagnostics

### 7.1 Why the sample covariance matrix isn't good enough — `covariance.py`

The plain sample covariance matrix,

$$\hat{\Sigma} = \frac{1}{T} \sum_{t=1}^T (x_t - \bar{x})(x_t - \bar{x})^\top$$

is unbiased, but it's a genuinely bad estimator whenever the number of assets  $N$  isn't much smaller than the number of observations  $T$  – which, for anything resembling a real equity universe (hundreds or thousands of names, a few years of daily data), is basically always the regime you're in. The symptom is a matrix with an enormous condition number (ratio of largest to smallest eigenvalue): the smallest eigenvalues are pushed toward zero by estimation noise, and a portfolio optimizer run against a matrix like that will happily load up on exactly those noisy low-variance directions, mistaking estimation error for genuine diversification.

### 7.2 Ledoit-Wolf shrinkage

The fix I implemented is Ledoit & Wolf's (2004) optimal linear shrinkage: blend the noisy sample covariance  $S$  with a more structured, lower-variance target  $F$ ,

$$\hat{\Sigma}_{\text{shrunk}} = \rho F + (1 - \rho)S$$

and choose  $\rho \in [0, 1]$  to minimize the expected squared (Frobenius norm) distance to the true, unobserved covariance matrix. The target I use by default is the “constant correlation” target from their original paper: keep the sample variances on the diagonal, but replace every pairwise correlation with the single average correlation across the whole matrix.

The optimal  $\rho$  works out to be a ratio of two quantities: how much sampling noise is baked into  $S$  (estimated from how much each period's individual outer product  $x_t x_t^\top$  disagrees with  $S$  on average), divided by how far  $S$  actually is from the target  $F$ :

```
1 distance_sq = np.sum((sample_cov - target_matrix) ** 2) / n_assets
2 outer_products = np.einsum("ti,tj->tij", centered, centered)
3 noise_sq = np.sum((outer_products - sample_cov) ** 2) / (n_obs ** 2 * n_assets)
4 noise_sq = min(noise_sq, distance_sq) # clip so rho can't exceed 1
5 shrinkage_intensity = 0.0 if distance_sq == 0 else noise_sq / distance_sq
6 shrunk_cov = shrinkage_intensity * target_matrix + (1.0 - shrinkage_intensity) * sample_cov
```

The intuition is straightforward once you see it this way: if the sample covariance is mostly noise (large `noise_sq` relative to `distance_sq`), shrink hard toward the stable target; if the sample estimate looks stable and reliable, don't shrink much.

### 7.3 PCA-based statistical factor model

The other standard fix is to stop estimating every pairwise covariance directly and instead explain the covariance structure through a small number of factors, the same  $\Sigma = X \Sigma_f X^\top + \Delta$  structure used in fundamental (Barra-style) factor models, except the factors here come from PCA rather than named industry/style exposures.

Eigendecompose the sample covariance, keep the top  $k$  eigenvalue/eigenvector pairs as the factor structure, and put whatever variance those  $k$  factors don't explain into a diagonal “specific risk” term:

$$\hat{\Sigma} = \underbrace{V_k \Lambda_k V_k^\top}_{\text{factor part}} + \underbrace{\text{diag}(\text{diag}(S) - \text{diag}(V_k \Lambda_k V_k^\top))}_{\text{specific risk}}$$

```

1 eigenvalues, eigenvectors = np.linalg.eigh(sample_cov)
2 order = np.argsort(eigenvalues)[::-1]
3 loadings = eigenvectors[:, order][:, :n_factors]
4 top_eigenvalues = eigenvalues[order][:n_factors]
5 factor_covariance_part = loadings @ np.diag(top_eigenvalues) @ loadings.T
6 specific_variance = np.clip(np.diag(sample_cov) - np.diag(factor_covariance_part), 0.0, None)
7 reconstructed = factor_covariance_part + np.diag(specific_variance)

```

`examples/run_covariance_estimation.py` simulates 60 assets from 80 observations with a known 3-factor true covariance (deliberately close to the  $N \approx T$  danger zone) and compares all three estimators. On one run: the raw sample covariance had a condition number around 8,300; Ledoit-Wolf shrinkage brought that down to about 630; the 3-factor PCA model (correctly specified, since the simulation really does have 3 factors) brought it down to about 330 – both a large, visible improvement over the raw sample estimate.

## 7.4 Eigenvalue diagnostics

A smaller helper, `eigenvalue_diagnostics`, just reports the smallest/largest eigenvalues and condition number of any covariance matrix. I want to be explicit that this is a diagnostic, not a correction: a large condition number is a warning sign (the “optimization bias” problem mentioned above), but properly correcting for it – Barra’s eigenfactor risk adjustment, which re-simulates each eigenfactor’s true variance via Monte Carlo – is out of scope here. This function tells you whether you should be worried; it doesn’t fix the problem for you.

## 7.5 ECDF, Q-Q plot, Jarque-Bera — `diagnostics.py`

A lot of the rest of the package leans on returns being roughly normal (parametric VaR, CAPM, Black-Scholes), and real returns reliably violate that to some degree, so I built the standard visual and statistical checks.

The Q-Q plot compares sorted sample quantiles against the quantiles a normal distribution with the same mean and standard deviation would produce, using the standard  $(i - 0.5)/n$  plotting-position convention:

```

1 plotting_positions = (np.arange(1, n + 1) - 0.5) / n
2 theoretical_quantiles = np.array([normal_ppf(p) for p in plotting_positions])
3 theoretical_values = mean + std * theoretical_quantiles

```

Points falling on the 45-degree line mean “looks normal”; systematic curvature at the ends is the classic fat-tails signature.

The Jarque-Bera statistic formally tests normality using sample skewness  $S$  and excess kurtosis  $K$ :

$$JB = \frac{n}{6} \left( S^2 + \frac{K^2}{4} \right) \sim \chi_2^2 \text{ under the null of normality}$$

```

1 skewness = float(np.mean(standardized ** 3))
2 excess_kurtosis = float(np.mean(standardized ** 4) - 3.0)
3 jarque_bera_stat = (n / 6.0) * (skewness ** 2 + (excess_kurtosis ** 2) / 4.0)
4 p_value = 1.0 - chi2_cdf_df2(jarque_bera_stat)

```

This is why `chi2_cdf_df2` from Section 2 exists – it’s the distribution of the JB statistic under the null.

**Run it:** `plot_ecdf` and `qq_plot` are exercised (headlessly, via `matplotlib.use("Agg")`) in `tests/test_diagnostics.py`; there isn't a standalone example script for these since they're small enough to call directly on any returns series you already have loaded.

## 8 Portfolio construction — quantfin/portfolio.py

### 8.1 Why everything here is closed-form

Rather than reaching for a general-purpose numerical optimizer (`scipy.optimize`, `cvxpy`), every objective in this module reduces to an equality-constrained quadratic program that I solve directly via its Lagrangian / KKT conditions – a couple of linear solves, no iteration, no convergence tolerance to tune. This only works because the sole constraint is the budget constraint  $\mathbf{1}^\top w = 1$  (fully invested, short-selling allowed); if you need long-only or position-limit constraints, that's an inequality-constrained QP and you'd need a real solver like `cvxpy`.

### 8.2 The one function everything reduces to

Every objective below is, after writing out its first-order conditions, the same problem: minimize  $\frac{1}{2}w^\top Aw - w^\top b$  subject to  $\mathbf{1}^\top w = 1$ . Forming the Lagrangian

$$\mathcal{L}(w, \gamma) = \frac{1}{2}w^\top Aw - w^\top b - \gamma(\mathbf{1}^\top w - 1)$$

and setting  $\partial\mathcal{L}/\partial w = 0$  gives  $Aw = b + \gamma\mathbf{1}$ , so  $w = A^{-1}(b + \gamma\mathbf{1}) = A^{-1}b + \gamma A^{-1}\mathbf{1}$ . Plugging back into the constraint  $\mathbf{1}^\top w = 1$  pins down  $\gamma$ :

$$\gamma = \frac{1 - \mathbf{1}^\top A^{-1}b}{\mathbf{1}^\top A^{-1}\mathbf{1}}$$

```

1 def _solve_equality_constrained_qp(A: np.ndarray, b: np.ndarray) -> np.ndarray:
2     ones = np.ones(A.shape[0])
3     A_inv_b = np.linalg.solve(A, b)
4     A_inv_ones = np.linalg.solve(A, ones)
5     gamma = (1.0 - ones @ A_inv_b) / (ones @ A_inv_ones)
6     return A_inv_b + gamma * A_inv_ones

```

### 8.3 Minimum variance and mean-variance (with transaction costs)

Minimum variance: minimize  $w^\top \Sigma w$  subject to the budget constraint – exactly the QP above with  $A = \Sigma$ ,  $b = 0$ .

Mean-variance with a risk aversion coefficient  $A_{\text{risk}}$ : maximize  $w^\top \mu - \frac{A_{\text{risk}}}{2} w^\top \Sigma w$ , which after negating (to turn it into a minimization) matches the QP with  $A = A_{\text{risk}}\Sigma$ ,  $b = \mu$ .

Transaction costs are modeled as a quadratic penalty on trade size relative to a starting portfolio  $w_0$ :  $\frac{\kappa}{2} \|w - w_0\|^2$ . This is a simplification of real trading costs (which are usually closer to piecewise-linear plus a fixed component), but it's the one functional form that keeps the whole problem quadratic and solvable this way – add  $\kappa I$  to  $A$  and  $\kappa w_0$  to  $b$ , and the same closed form applies unchanged:

```

1 elif objective == "mean_variance":
2     a_matrix = risk_aversion * sigma + transaction_cost_coefficient * np.eye(n)
3     b_vector = mu + (transaction_cost_coefficient * current_weights
4                     if transaction_cost_coefficient > 0 else 0.0)
5     weights = _solve_equality_constrained_qp(a_matrix, b_vector)

```

In `examples/run_portfolio_optimization.py` I check this does what it should: solving with a nonzero transaction cost coefficient against a fixed starting allocation pulls the optimal weights visibly back toward the starting point compared to the frictionless solution – exactly the tradeoff you'd expect between chasing the unconstrained optimum and not paying to trade all the way there.

## 8.4 Maximum Sharpe ratio (tangency portfolio)

The tangency portfolio has its own, slightly different closed form (it doesn't fit the QP-with-transaction-costs template above, which is also why transaction costs aren't supported for this objective):

$$w^* = \frac{\Sigma^{-1}(\mu - r_f \mathbf{1})}{\mathbf{1}^\top \Sigma^{-1}(\mu - r_f \mathbf{1})}$$

the numerator is a raw (unnormalized) mean-variance-efficient direction, and the denominator just rescales it back onto the budget constraint.

```

1 excess_returns = mu - risk_free_rate
2 inv_sigma_excess = np.linalg.solve(sigma, excess_returns)
3 denominator = np.ones(n) @ inv_sigma_excess
4 weights = inv_sigma_excess / denominator

```

## 8.5 Efficient frontier

Tracing the frontier means solving the same QP repeatedly for a range of target returns, but there's a closed form for the whole family at once. Define

$$A = \mathbf{1}^\top \Sigma^{-1} \mathbf{1}, \quad B = \mathbf{1}^\top \Sigma^{-1} \mu, \quad C = \mu^\top \Sigma^{-1} \mu, \quad D = AC - B^2$$

Then for a target return  $\mu_p$ , the minimum-variance portfolio hitting that target is

$$w = \Sigma^{-1} [\gamma_1 \mathbf{1} + \gamma_2 \mu], \quad \gamma_1 = \frac{C - B\mu_p}{D}, \quad \gamma_2 = \frac{A\mu_p - B}{D}$$

which is a standard result from writing out the Lagrangian with *two* constraints (budget *and* target return) instead of one – I derived it the same way as the single-constraint case above, just carrying two multipliers through instead of one.

```

1 for target in target_returns:
2     gamma_1 = (c_coef - b_coef * target) / determinant
3     gamma_2 = (a_coef * target - b_coef) / determinant
4     weights = np.linalg.solve(sigma, gamma_1 * ones + gamma_2 * mu)

```

## 8.6 Risk decomposition: MCTR and CCTR

Given a portfolio's weights, how much does each position actually contribute to total portfolio volatility? Portfolio volatility is  $\sigma_p = \sqrt{w^\top \Sigma w}$ . Differentiating with respect to a single weight  $w_i$  gives the Marginal Contribution to Risk:

$$\text{MCTR}_i = \frac{\partial \sigma_p}{\partial w_i} = \frac{(\Sigma w)_i}{\sigma_p}$$

Because  $\sigma_p(w)$  is homogeneous of degree 1 in  $w$  (scale every weight by  $\lambda$  and volatility scales by  $\lambda$  too), Euler's homogeneous function theorem says

$$\sigma_p = \sum_i w_i \cdot \frac{\partial \sigma_p}{\partial w_i} = \sum_i w_i \cdot \text{MCTR}_i$$

so defining Component Contribution to Risk as  $\text{CCTR}_i = w_i \cdot \text{MCTR}_i$  gives a decomposition where every position's contribution sums *exactly* to total portfolio volatility – which is what makes it useful for a risk report where the numbers actually need to add up.

```
1 def marginal_contribution_to_risk(weights, covariance):
2     w = np.asarray(weights, dtype=float)
3     portfolio_volatility = np.sqrt(w @ covariance @ w)
4     return (covariance @ w) / portfolio_volatility
5
6 def component_contribution_to_risk(weights, covariance):
7     return weights * marginal_contribution_to_risk(weights, covariance)
```

I check the Euler identity directly in `tests/test_portfolio.py` – `cctr.sum()` against `portfolio_volatility` to within floating point tolerance – and again at the end of `run_portfolio_optimization.py` on a live example, where the sum of contributions matches the reported portfolio volatility to four decimal places.

**Run it:**

```
PYTHONPATH=. python examples/run_portfolio_optimization.py
```

## 9 Risk and model validation — quantfin/risk/

This is the part of the package I think about as the actual point of the exercise: it's one thing to compute a VaR number, and a different thing to know how you'd check whether that VaR number is any good. Everything in this section is written with that distinction in mind.

### 9.1 Value at Risk and Expected Shortfall — value\_at\_risk.py

**Sign convention**, stated once here because it matters: every function in this module returns VaR/ES as a *positive* number representing the size of a loss (0.02 means “a 2% loss”), which is the convention desks actually use day to day, even though it means the formulas have their signs flipped relative to how you'd write them down reporting the raw P&L level. I'm explicit about this in the module docstring because a version of exactly this sign confusion was present in the tutorial script I originally started this module from – it's a genuinely easy place to get backwards.

**Historical VaR** makes no distributional assumption: it's just the empirical loss at the given percentile of observed returns.

$$\text{VaR}_\alpha^{\text{hist}} = -Q_{1-\alpha}(\text{returns})$$

```
1 loss_quantile = np.percentile(returns, (1.0 - confidence_level) * 100.0)
2 return float(-loss_quantile)
```

**Parametric VaR** assumes returns are normally distributed with the sample mean  $\mu$  and standard deviation  $\sigma$ :

$$\text{VaR}_\alpha^{\text{param}} = \sigma\sqrt{h} \cdot z_\alpha - \mu h$$

where  $z_\alpha = \Phi^{-1}(\alpha)$  and  $h$  is the horizon in days (the  $\sqrt{h}$  scaling on volatility is the standard square-root-of-time rule, which follows from variance being additive across independent periods).

```
1 z = normal_ppf(confidence_level)
2 return std * math.sqrt(horizon_days) * z - mean * horizon_days
```

**Expected Shortfall** (a.k.a. CVaR) is the average loss *given that* the loss exceeds VaR – unlike VaR itself, this actually says something about how bad the tail is, rather than just where the tail starts. Historically, it's the mean of the observed losses beyond the empirical VaR threshold; parametrically, for a normal distribution it has a closed form via the truncated-normal mean identity:

$$\text{ES}_\alpha^{\text{param}} = \sigma \cdot \frac{\phi(z_\alpha)}{1 - \alpha} - \mu$$

```
1 if method == "parametric":
2     z = normal_ppf(confidence_level)
3     return std * normal_pdf(z) / (1.0 - confidence_level) - mean
```

**Monte Carlo VaR** is the same idea as parametric VaR but reached via simulation instead of the closed form – deliberately, since for a genuinely normal model it should converge to the same answer as the parametric formula (a useful cross-check), and the payoff is that swapping in a richer return model later (fat tails, jumps) wouldn't require changing how VaR is read off the simulated losses.

### 9.2 VaR backtesting — backtesting.py

Computing a VaR number is easy. Knowing whether it's a *good* VaR number is the actual model-validation problem, and it's what turns “we have a risk model” into something a regulator (or an interviewer) can actually interrogate.



**Kupiec’s proportion-of-failures test.** If a 99% VaR model is correctly calibrated, exceptions (days the realized loss exceeds VaR) should occur with probability  $p = 1 - \alpha = 0.01$  each day, independently. Over  $n$  days with  $x$  observed exceptions, this is a simple likelihood ratio test of  $H_0 : p = p_0$  against  $H_1 : p = \hat{p} = x/n$ :

$$LR_{\text{POF}} = -2 \ln \left[ \frac{(1 - p_0)^{n-x} p_0^x}{(1 - \hat{p})^{n-x} \hat{p}^x} \right] \sim \chi_1^2 \text{ under } H_0$$

```

1 log_likelihood_null = (n_obs - exceptions) * _safe_log(1 - expected_rate) \
2   + exceptions * _safe_log(expected_rate)
3 log_likelihood_alt = (n_obs - exceptions) * _safe_log(1 - observed_rate) \
4   + exceptions * _safe_log(observed_rate)
5 statistic = max(-2.0 * (log_likelihood_null - log_likelihood_alt), 0.0)
6 p_value = 1.0 - chi2_cdf_df1(statistic)

```

A large statistic (small p-value) means the observed exception rate is statistically inconsistent with the model’s stated confidence level – the model is miscalibrated, either too conservative or not conservative enough.

**Christoffersen’s independence test.** Kupiec only checks the *count* of exceptions is right; it says nothing about whether they’re clustered in time, which matters because clustering suggests the model is slow to react to changing volatility (exactly the failure mode that mattered in real crises). This test fits a first-order Markov chain to the 0/1 exception sequence and compares a restricted model (exception probability independent of yesterday’s state) against an unrestricted one (exception probability allowed to depend on it):

$$LR_{\text{ind}} = -2 \ln \left[ \frac{\mathcal{L}(\text{restricted})}{\mathcal{L}(\text{unrestricted})} \right] \sim \chi_1^2$$

```

1 n00, n01, n10, n11 = 0, 0, 0, 0
2 for previous, current in zip(indicator[:-1], indicator[1:]):
3     if previous == 0 and current == 0: n00 += 1
4     elif previous == 0 and current == 1: n01 += 1
5     elif previous == 1 and current == 0: n10 += 1
6     else: n11 += 1
7 # restricted: single pooled exception probability
8 # unrestricted: separate probabilities conditional on yesterday’s state

```

**Conditional coverage test.** Add the two LR statistics together – under the null, the sum of two independent  $\chi_1^2$  statistics is  $\chi_2^2$  – giving a single test that checks both correct coverage *and* independence at once:

```

1 statistic = coverage_result.statistic + independence_result.statistic
2 p_value = 1.0 - chi2_cdf_df2(statistic)

```

**Bias statistic.** A rolling check on volatility *forecasts* rather than VaR exceptions directly: standardize realized returns by the model’s predicted volatility, then take the rolling standard deviation of the standardized series. If the volatility model is well calibrated, this should sit close to 1.0; persistently above 1 means the model under-forecasts risk, persistently below means it over-forecasts.

```

1 standardized_returns = realized / predicted
2 for t in range(window - 1, n):
3     segment = standardized_returns[t - window + 1 : t + 1]
4     result[t] = np.std(segment, ddof=1)

```

**Run it:**

```
PYTHONPATH=. python examples/run_var_backtesting.py
```

This script simulates returns with a bit of volatility clustering, runs a rolling 250-day parametric VaR backtest against them, and prints all four diagnostics together – which is closer to what an actual model validation writeup would look like than any single test in isolation.

### 9.3 CAPM — capm.py

The Capital Asset Pricing Model says an asset's expected excess return is proportional to its exposure to systematic (market) risk:

$$E[r_i] - r_f = \beta_i (E[r_m] - r_f), \quad \beta_i = \frac{\text{Cov}(r_i, r_m)}{\text{Var}(r_m)}$$

I implemented  $\beta$  two independent ways precisely so I could check they agree: directly from the covariance/variance ratio, and via ordinary least squares regression of asset returns on market returns. These have to agree numerically, because the covariance-ratio formula for  $\beta$  is the OLS slope coefficient for a single-regressor regression – it's the same object derived two different ways, not two different estimators.

```

1 def beta_from_covariance(self) -> float:
2     covariance_matrix = np.cov(self.asset_returns, self.market_returns, ddof=1)
3     return float(covariance_matrix[0, 1] / covariance_matrix[1, 1])
4
5 def beta_from_regression(self) -> Tuple[float, float]:
6     beta, alpha = np.polyfit(self.market_returns, self.asset_returns, deg=1)
7     return float(beta), float(alpha)

```

The required risk premium for an investor with (absolute) risk aversion coefficient  $A$  holding a position with return variance  $\sigma^2$ , under the standard mean-variance / quadratic-utility approximation, is

$$\Pi = \frac{1}{2} A \sigma^2$$

which is the same risk-aversion parameter that shows up in `portfolio.optimize_portfolio`'s "mean\_variance" objective – it's what ties an investor's risk tolerance directly to how much expected return they need to be compensated for taking on a given amount of variance.

**Run it:**

```
PYTHONPATH=. python examples/run_capm_analysis.py
```

## 10 Running everything and the test suite

### 10.1 Example scripts, all in one place

| Script                                     | What it demonstrates                                                                             |
|--------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>run_bond_pricing.py</code>           | Zero-coupon and coupon bond pricing, YTM, duration, convexity                                    |
| <code>run_callable_bond_pricing.py</code>  | Callable/puttable bonds through the short-rate lattice, with a fair plain-bond baseline          |
| <code>run_option_pricing.py</code>         | Black-Scholes vs. binomial tree vs. Monte Carlo, Greeks, American vs. European put               |
| <code>run_fdm_pricing.py</code>            | Explicit/implicit/Crank-Nicolson PDE solvers, plus an intentional instability demo               |
| <code>run_covariance_estimation.py</code>  | Sample covariance vs. Ledoit-Wolf vs. PCA factor model on a simulated near-singular panel        |
| <code>run_portfolio_optimization.py</code> | Efficient frontier, three optimization objectives, transaction-cost-aware rebalancing, MCTR/CCTR |
| <code>run_var_backtesting.py</code>        | VaR/ES estimation and a full rolling Kupiec/Christoffersen/bias-statistic backtest               |
| <code>run_capm_analysis.py</code>          | CAPM beta (two ways), expected return, risk-aversion-implied premium                             |

Every one of them is run the same way, from the repository root:

```
PYTHONPATH=. python examples/<script_name>.py
```

(or, after `pip install -e .`, just `python examples/<script_name>.py` without the `PYTHONPATH` prefix).

### 10.2 Testing methodology

142 tests across 15 files, run with the standard library’s `unittest` (no `pytest` dependency):

```
python -m unittest discover -s tests -v
```

I tried to make sure these are real checks and not just “does it run without crashing”:

- **Known closed-form identities:** put-call parity for the option pricers, Euler’s theorem for MCTR/CCTR, the forward-rate telescoping identity for the bond lattice.
- **Hand-calculated reference values:** e.g.  $z_{0.95} = 1.6449$  and  $\chi^2_{1,0.95} = 3.841$  checked directly against the numerics module.
- **Round-trip checks:** computing a bond’s YTM from a price, then recomputing the price from that YTM, should return the original price.
- **Statistical convergence checks:** Monte Carlo option prices should land within a few standard errors of the closed-form Black-Scholes price.
- **Cross-method agreement:** CAPM beta from the covariance formula vs. from OLS regression; Black-Scholes vs. binomial tree vs. Monte Carlo, all on the same option.

Two real bugs got caught this way during development, both described in-line in Sections 5 and 5.5: the accrued-interest off-by-one on coupon dates, and an early test I’d written incorrectly for the

American option exercise clamp (which turned out to be a mistake in the test's expected value, not the pricing code – backward induction discounts an already-clamped continuation value by one more step, so a clamp doesn't collapse a whole subtree to exactly the clamp price the way I'd first assumed).